Cost-Aware Routing of Large Language Models: Predicting the Cheapest Capable Model for Each Request

Michael Valderrama

Shiley-Marcos School of Engineering, University of San Diego

AAI-501-01: Introduction to Artificial Intelligence and Machine Learning

Andrew Van Benschoten, Ph.D.

August 2026

Code repository: https://github.com/ibucketbranch/MS-AAI-501-Final_Project_IntroAI

Purpose, Goals, and Scope

This project asked a budgeting question about large language models: when a request arrives, which model should get it? Sending every request to the strongest available model is the safe default and also the expensive one. For many requests a cheaper model gives an answer that scores just as well, so I built and compared machine learning models that act as an accountant for token spend, picking the cheapest model expected to handle each request adequately, and I measured what that routing actually earns against baselines that are allowed to win.

I worked from LLMRouterBench (Li et al., 2026), a public benchmark that records how 13 flagship models answered the same prompts, along with the tokens used, the dollar cost, and a quality score for each answer. Flattening the release files produced 173,688 prompt-model records across 12,166 usable prompts spanning nine task datasets including factual recall, competition math, and software engineering. The benchmark also ships per-prompt results for OpenRouter, a commercial routing service, which I set aside as a reference strategy and never used in training.

The routing label came from the recorded data rather than from my judgment. For each prompt, the cost-optimal model is the cheapest one whose score clears a quality threshold. I used a threshold of 1.0 as the primary setting and re-ran the analysis at 0.5 to test how sensitive the conclusions are to that choice. Framing the label this way gave me two supervised tasks straight from the course material: classification, predicting the cost-optimal model from light pre-call features, and regression, predicting what a request will cost before committing to it. I compared logistic regression against a tuned random forest on the classification task, built a random forest cost regressor for the second, and scored every strategy on a held-out test set against always-cheapest, always-strongest, the best fixed single model, an oracle upper bound, and the OpenRouter reference.

I adapted published framings rather than devising a new algorithm. The label construction follows the routing literature (Ong et al., 2024), and the evaluation setting comes from the benchmark itself. What I added was comparison discipline: prompt-level splits so no prompt sits

on both sides of train and test, sensitivity analysis on the threshold, and success measured as cost saved at a given quality level rather than accuracy alone, since accuracy does not capture money. The methods map to the supervised learning and model evaluation material in the course (Poole & Mackworth, 2023, Chapter 7): two algorithm families, hyperparameter tuning by cross-validation, feature importance, and an experimental comparison presented graphically.

I set two scope limits deliberately. Features stayed light, prompt length, token counts, and task category, with no text embeddings, so the results measure what cheap pre-call signals can do on their own. And everything ran as offline replay against recorded outcomes; the routers never called a live model, which keeps the comparison controlled and repeatable.

Data and the Routing Label

LLMRouterBench's Performance-Cost setting evaluates 13 flagship models on 10 source datasets covering mathematics (aime, livemathbench), code (livecodebench, swe-bench), scientific and general reasoning (gpqa, hle, mmlupro), factual recall (simpleqa), and open-ended instruction following (arenahard). Each record pairs one prompt with one model and carries the prompt and completion token counts, the dollar cost of that response, and a quality score. Nine of the ten datasets score answers 0 or 1; arenahard also awards 0.5 for a draw against a reference answer.

I flattened the released JSON files into one long table of 173,688 prompt-model records covering 12,446 prompts. The analysis needed the full 13-model menu for every prompt, which forced two exclusions: the tau2 dataset dropped out entirely because one model, gpt-5-chat, was never run on it, and two prompts whose query text did not align across models were removed. The working sample was 12,166 prompts across nine datasets. Everything is measured public data; my preprocessing was reshaping, not cleaning.

The routing label was the highest-risk design decision in the project, so I wrote it down as a rule and tested it. For each prompt, the label is the cheapest model whose recorded score clears the quality threshold. At the primary threshold of 1.0 the labeled model actually solved the prompt. For 18 percent of prompts no model cleared the bar; there the label falls back to the cheapest model overall, on the reasoning that when every answer fails, the only objective left is

minimizing spend. The sensitivity threshold of 0.5, which additionally accepts arenahard draws as capable, relabeled 2.1 percent of prompts. Section 4 shows the conclusions did not move.

I restricted the features to information available before any model is called: the prompt's character length, the median prompt-token count across the model pool as a tokenizer-neutral size measure, and the task category. I made the 80/20 split at the prompt level, stratified by dataset. Because the modeling table holds one row per prompt, no prompt can appear on both sides of the split, which avoids the leakage a row-level split of the long table would have created, since the same prompt appears there thirteen times.

Algorithms: Theory and Specification

The classification task is multiclass: given a prompt's features, predict which of the 13 models is cost-optimal. I trained two algorithm families on it and a third family on the regression task, so the required experimental comparison runs both within a task and across strategies.

Multinomial logistic regression models the probability of each class as a softmax over linear scores: for class k, $P(y = k \mid x)$ is proportional to $\exp(w_k \cdot x + b_k)$. Training maximizes the regularized log-likelihood, and the inverse regularization strength C controls how hard the weights are penalized. It is the interpretable baseline: one weight per feature per class, and a linear decision boundary. I tuned it with 5-fold cross-validation on the training prompts, selecting by macro F1 rather than accuracy so that rare but valuable classes, the prompts only frontier models can solve, are not drowned out by the majority classes. The search chose C = 10 with no class weighting; balanced class weights lost the search.

The random forest (Breiman, 2001) is the non-linear comparison. It grows an ensemble of decision trees, each on a bootstrap sample of the training data with a random subset of features considered at each split, and averages their votes. The bootstrapping and feature randomness decorrelate the trees, so the ensemble cuts variance without the single-tree tendency to overfit. The same cross-validation search selected 300 trees with unrestricted depth, again with no class weighting. The forest also supplies impurity-based feature importances, shown in Figure 3.

The second required algorithm type is regression: predict the dollar cost of a specific prompt-model call from the same pre-call features plus the model identity. Costs span four orders of magnitude and 2,226 records cost exactly zero, so I predicted log10(cost + 0.0001) instead of raw dollars. A tuned random forest regressor (depth 12, minimum leaf size 2) reached R-squared of 0.79 on the log scale, with a dollar-scale mean absolute error of $0.0073 against an average per-call bill of $0.014.

The regressor became a third routing strategy by pairing it with a capability screen computed on training data only: a model counts as capable on a task category if it solved at least half of that category's training prompts. Each test prompt is routed to the cheapest predicted-price capable model, falling back to the cheapest model on the two categories where no model qualified (hle and swe-bench). This turns the cost model into a router without ever letting test information leak into the decision rule.

All code is Python (scikit-learn pipelines in documented Jupyter notebooks), follows PEP 8 under a ruff lint workflow in continuous integration, and includes unit tests on the label rule. The public repository link is on the title page.

Experimental Results

Every strategy was scored on the same 2,434 held-out test prompts. Table 1 reports the mean dollar cost and mean quality score per prompt; Figure 1 plots the same strategies on the cost-versus-quality plane alongside all 13 fixed models, with cost on a log scale.

Table 1. Strategy comparison on the held-out test set (threshold 1.0).

Strategy	**Mean cost (USD)**	**Mean score**
Regressor-derived router	0.0006	0.513
Always cheapest (deepseek-v3-0324)	0.0008	0.429
Best single value model (qwen3-235b-a22b-2507, fixed)	0.0009	0.538
Oracle label (ceiling)	0.0055	0.822

Strategy	Mean cost (USD)	Mean score
Random forest router (tuned)	0.0073	0.536
Logistic router (tuned)	0.0115	0.564
OpenRouter (reference)	0.0225	0.495
Always strongest (gemini-2.5-pro)	0.0615	0.597

The spread is wide for strategies consuming identical traffic. The logistic router achieved the highest quality among the trained strategies, 0.564, which recovers 94 percent of the always-strongest score at 19 percent of its cost. The random forest router landed close to the fixed qwen model on quality at eight times its price. The regressor-derived router produced the cheapest routing on the chart, undercutting even the always-cheapest fixed model, because no single model is the cheapest on every prompt; it bought that economy with a quality discount.

Figure 1

Cost versus quality on the held-out test prompts: every routing strategy and each of the 13 models as a fixed strategy (cost on a log scale).

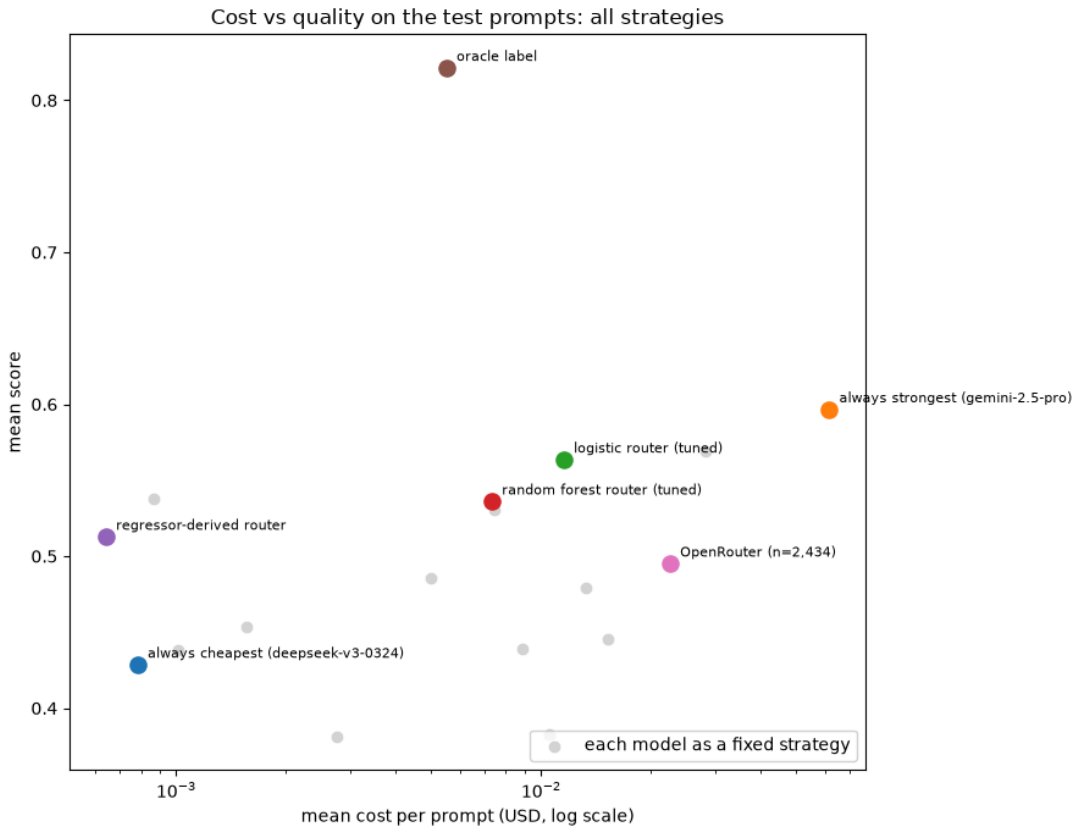

Figure 1. Cost versus quality on the held-out test prompts: every routing strategy and each of the 13 models as a fixed strategy (cost on a log scale).

The confusion matrix of the winning classifier (Figure 2, classes ordered cheap to expensive) shows the errors are not random: most confusion happens between adjacent cheap models, where the price difference is small, and the model rarely mistakes a cheap-solvable prompt for a frontier-only one. The forest's feature importances (Figure 3) rank task category first by a wide margin, then prompt length. That matches the exploratory picture from Figure 4: which model solves a prompt depends far more on what kind of task it is than on how long it is.

Figure 2

Confusion matrix of the tuned logistic router on the test prompts, classes ordered cheapest to most expensive.

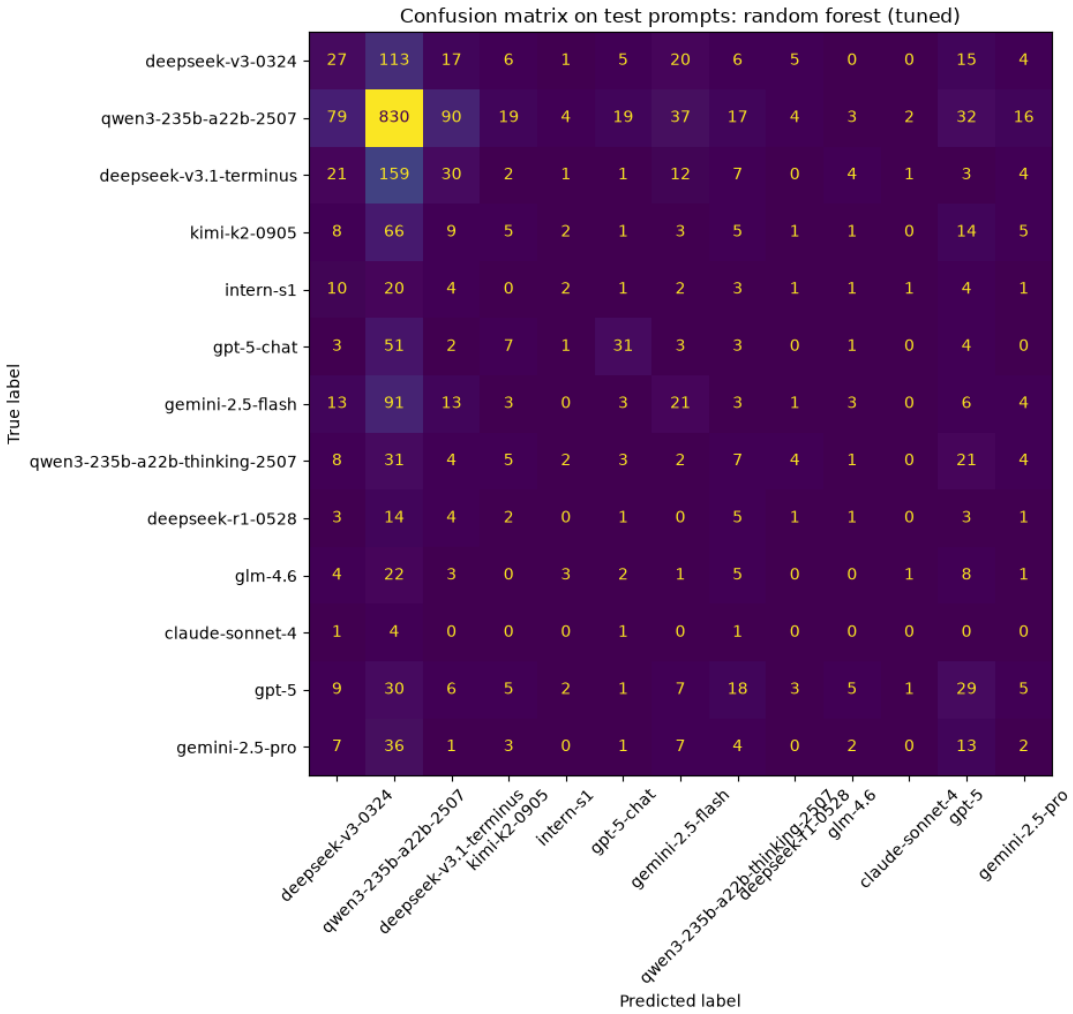

Figure 2. Confusion matrix of the tuned logistic router on the test prompts, classes ordered cheapest to most expensive.

Figure 3

Impurity-based feature importances from the tuned random forest router.

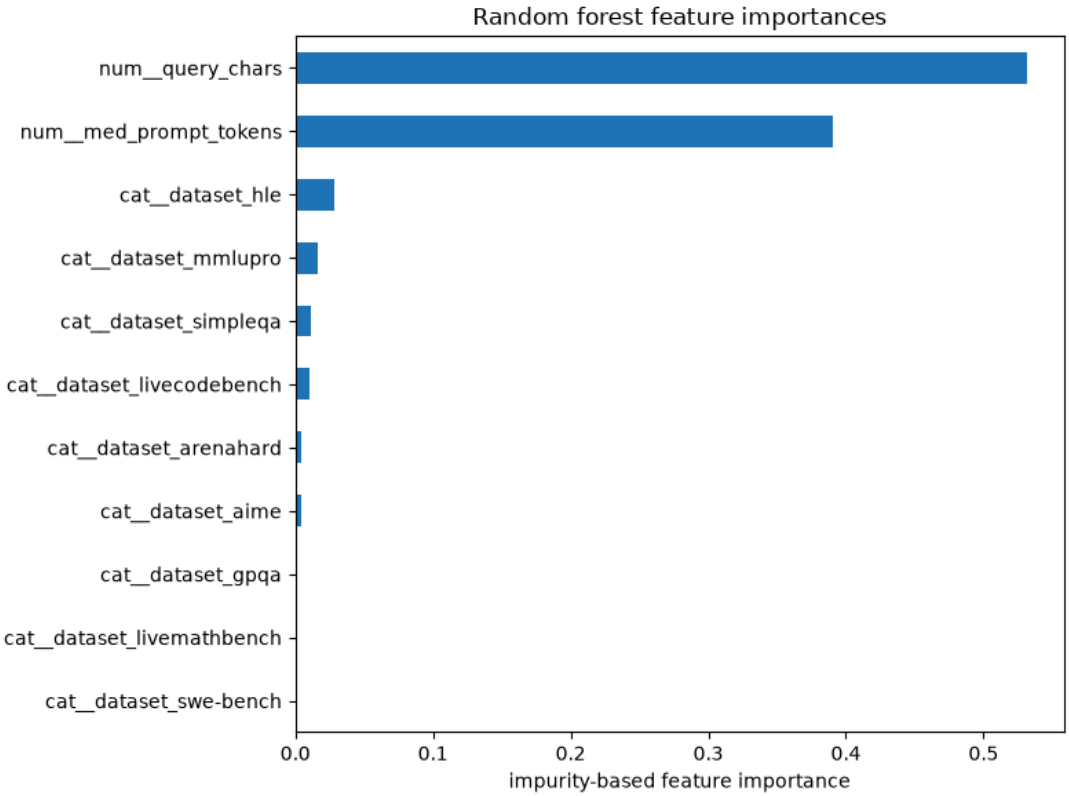

Figure 3. Impurity-based feature importances from the tuned random forest router.

Figure 4

Mean score per model per source dataset (rows ordered cheap to expensive): task category separates the models far more than prompt size does.

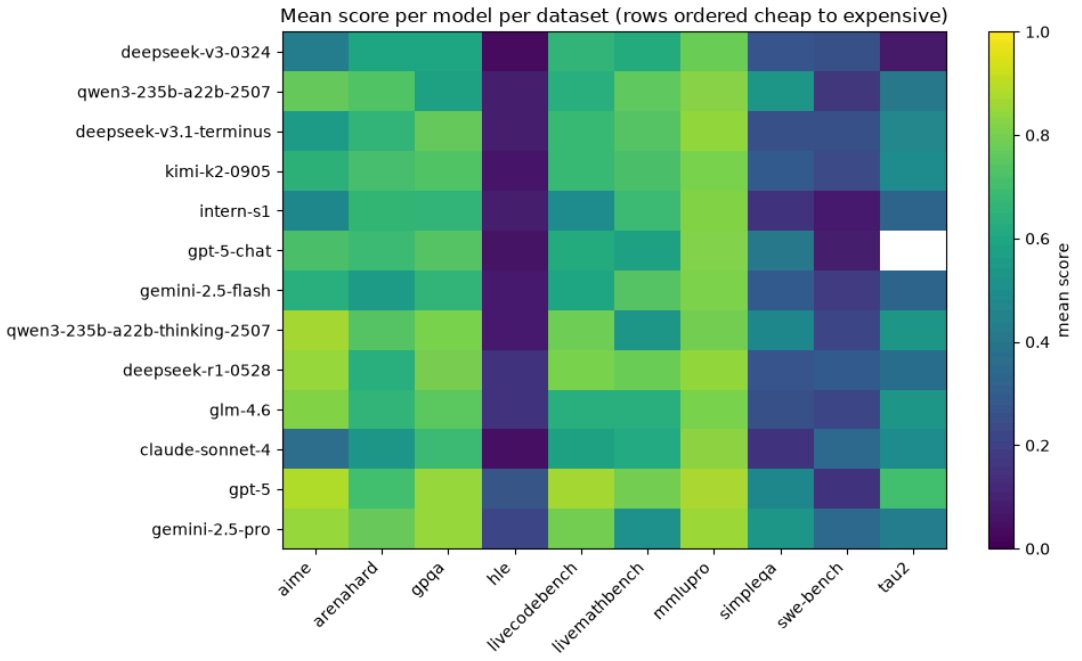

Figure 4. Mean score per model per source dataset (rows ordered cheap to expensive): task category separates the models far more than prompt size does.

Rebuilding the labels at threshold 0.5 relabeled 2.1 percent of prompts. Retrained on those labels, the logistic router's test numbers did not move at the reported precision, the random forest shifted by $0.0003 in cost and 0.002 in score, and the oracle ceiling dipped from 0.822 to 0.813. No strategy ranking changed, so the threshold-1.0 results carry the report.

Perfect routing would score 0.822 at $0.0055 per prompt, cheaper than the tuned random forest actually ran. Near-perfect routing is not expensive; the difficulty is informational, not economic. The gap between the trained routers and the oracle is the price of predicting from pre-call features alone.

Discussion

Two comparisons discipline these results. The first is the best fixed single model. Sending every prompt to qwen3-235b-a22b-2507 scored 0.538 at $0.0009, which dominates the tuned random forest outright and out-scores the regressor router. A router has to beat the best single model to justify its existence, and on these features only the logistic router clears that bar on quality, at a price premium. The second is the commercial reference: OpenRouter was beaten on both cost

and quality by all three trained strategies, which surprised me for a pipeline built from course tools and three light features.

The pattern behind the first comparison is the finding I did not expect when I proposed this project. I predicted the baselines would sit at the two ends of the cost-quality tradeoff with the learned router in the useful middle, and that happened, but the real competitor turned out to be neither baseline. One cheap model is simply strong enough that a constant policy rivals per-request intelligence. The benchmark's authors report the same thing at full scale: several routers, including commercial ones, fail to outperform the best single model (Li et al., 2026). My study reproduced that published pattern independently, with different tools, at course scale. The state of the art closes the gap, their Avengers-Pro router sits essentially on the Pareto frontier of this setting with up to a 4 percent performance gain over the best single model or a 31.7 percent cost saving at held accuracy, but it does so with substantially heavier inputs, query embeddings and clustered model-expertise profiles, than the deliberately light feature set used here. That is the trade my scope limit bought: cheap signals get most of the economics and stop short of the ceiling.

The instructor suggested one stretch experiment that turned into the most memorable result. Instead of a trained classifier, I handed the routing decision to an LLM (claude-haiku-4.5): for each of 300 stratified test prompts, the router model received the request text and a menu of the 13 models with their training-set average cost and solve rate, and replied with a model name. On the matched sample the LLM router landed at $0.0011 mean cost and 0.502 mean score, beating the regressor router on quality and the tuned classifiers on cost, with two caveats. Its own inference cost, roughly $0.0008 per request at these prompt lengths, about doubles its effective cost, and it barely routed: 95 percent of its traffic went to the same qwen model the fixed strategy uses (in 13 of 300 replies it started solving the prompt instead of naming a model and fell back to the cheapest option). Three different approaches, a trained classifier, a cost regressor with a capability screen, and a prompted LLM, converged on the same conclusion: the hard part of routing turned out to be knowing the menu, not picking per request.

The limitations are the flip side of the design choices. Everything rests on one benchmark's recorded runs, in English, scored mostly binary. Offline replay means the routers were never

exposed to live traffic, latency, or model version drift. The feature set excluded embeddings on purpose, so these results say what cheap pre-call signals can do, not what routing can do at its best. And the label's fallback rule for unsolvable prompts, cheapest model wins, bakes an economic judgment into 18 percent of the training signal; the threshold sensitivity check limits but does not eliminate that concern.

Conclusion

The question was whether a learned router can pick the cheapest capable model for each request. The answer from this study is a qualified yes with a sharper lesson attached. Learned routing works: the best trained router held 94 percent of the strongest model's quality at a fifth of its cost, every trained strategy beat the commercial reference on both axes, and the conclusions survived a threshold sensitivity check. But the evaluation also showed that on this benchmark a single well-chosen cheap model captures most of that value with no machine learning at all, a pattern the benchmark's own authors report at full scale. The oracle ceiling, 0.822 at half a cent per prompt, says the remaining gap is informational rather than economic, and the published state of the art crosses it with query embeddings. That is the natural next experiment: keep the same evaluation frame and test whether embedding-based features close the gap that pre-call signals cannot. For a team running LLMs at scale, the practical takeaway from this project is to measure the menu before investing in the router, because the accountant's first finding was that one line item does most of the work.

References

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32. https://doi.org/10.1023/A:1010933404324

Li, H., Zhang, Y., Guo, Z., Wang, C., Tang, S., Zhang, Q., Chen, Y., Qi, B., Ye, P., Bai, L., Wang, Z., & Hu, S. (2026). LLMRouterBench: A massive benchmark and unified framework for LLM routing. Findings of the Association for Computational Linguistics: ACL 2026. https://arxiv.org/abs/2601.07206

Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., & Stoica, I. (2024). RouteLLM: Learning to route LLMs with preference data. arXiv. https://arxiv.org/abs/2406.18665

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Poole, D. L., & Mackworth, A. K. (2023). Artificial intelligence: Foundations of computational agents (3rd ed.). Cambridge University Press.

Appendix A: Contributions

This project was completed solo, with instructor approval, after my assigned teammate switched programs. I performed all phases of the work: topic selection and dataset fit-check, the proposal, repository and CI setup, data acquisition and reshaping, exploratory analysis, label design and sensitivity testing, feature engineering, model training and tuning for both algorithm types, the strategy comparison, the LLM-as-router experiment, all figures, and this report.

Appendix B: AI Tool Disclosure

I used generative AI tools (Claude, via the Claude Code environment) to aid my understanding of the course material and to accelerate the mechanical parts of this project. Concretely, the AI helped scaffold the data-loading script and notebook structure, executed training and evaluation runs I specified, checked my routing-label logic against edge cases, and helped structure early drafts of the methods and results text. I treated the AI as a thinking partner rather than an answer machine; it surfaced questions I would not have asked alone, including the leakage risk in a row-level split and the need for a threshold sensitivity check. Per the course's academic integrity guidance, AI tool use is acceptable when disclosed and accompanied by genuine understanding of the submitted content. I have reviewed, modified, and verified all AI-assisted text and code against the cited sources and the executed notebooks, and the methodology, claims, and conclusions presented here reflect my own understanding of the material. The cost-aware routing

framing, the accountant-for-tokens problem statement, the label design with its fallback rule, and the interpretation of the results are my own. Tools used: Claude (Anthropic), including Claude Code; Gemini (Google) for the initial LLM-as-router attempt before quota limits forced a switch to a paid Claude model.